

AEO and GEO

Storefront Next ships built-in support for Answer Engine Optimization (AEO) and Generative Engine Optimization (GEO) on the Product Detail Page (PDP) and Product Listing Page (PLP). AEO makes product and category data machine-readable so search engines, voice assistants, and AI-powered answer surfaces can accurately surface facts—name, price, availability, brand—without guessing from unstructured HTML. GEO supplies clear entity structure and aligned metadata so generative and retrieval-augmented AI systems can ground their responses in your storefront's authoritative data. When a user asks an AI assistant about a product or category you sell, GEO increases the likelihood that the AI cites your storefront as the source.

AEO and GEO have these benefits:

- Rich results in search engines: Structured Product data enables Google rich snippets: price, availability, and review stars inline in search results.
- AI citation and grounding: Clear entity graphs (products, offers, breadcrumbs) give Large Language Models (LLMs) and Retrieval-Augmented Generation (RAG) systems verifiable facts to cite rather than fabricated summaries.
- Voice and assistant surfaces: Answer engines can resolve who, what, how much, and in-stock queries directly from your structured data.
- Multisite consistency: URL helpers respect site/locale path prefixes and proxy headers, so structured data links are correct across all your storefronts without manual configuration.
- Zero page-breakage risk: Schema generation runs in a deferred Promise. If it fails, the page renders normally with no script tag or broken pages.

How It Works

Two technical mechanisms work together: JSON-LD and SeoMeta .

Mechanism	Purpose	V
JSON-LD (<code><script type="application/ld+json"></code>)	Structured entity data for crawlers and AI systems	P r e c e d e n t
SeoMeta	Classic <code><title></code> , <code><meta</code>	f (l



JSON-LD is the primary AEO and GEO signal. `SeoMeta` complements it—crawlers and social previews still read `<head>` tags even when structured data is present.

Product Detail Page (PDP)

What Gets Emitted

The PDP emits a `schema.org/Product` graph with these fields populated from B2C Commerce API (SCAPI) data when available.

Field	Source
<code>name</code>	<code>product.name</code>
<code>description</code>	<code>longDescription</code> , falling back to <code>shortDescription</code> , then <code>pageDescription</code> if earlier fields are empty
<code>image</code>	Up to 5 large or medium images (thumbnails excluded)
<code>sku</code> , <code>productID</code>	<code>product.id</code>
<code>url</code>	Public storefront origin + current path
<code>brand</code>	<code>product.brand</code>
<code>mpn</code>	<code>product.manufacturerSKU</code>
<code>gtin</code>	<code>product.ean</code>
<code>category</code>	<code>product.primaryCategoryId</code>
<code>color</code>	First <code>color</code> variation attribute value
<code>additionalProperty</code>	All variation attributes + custom attributes as <code>PropertyValue</code>
<code>offers.price</code>	Effective price (base or promo)
<code>offers.priceCurrency</code>	<code>product.currency</code>
<code>offers.availability</code>	Derived from inventory (<code>InStock</code> , <code>OutOfStock</code> , <code>BackOrder</code> , and <code>PreOrder</code>)



`offers.lowPrice` and
`highPrice`

Populated for master products
with a price range across variants

`SeoMeta` sets `<title>` to the product name, meta description to `pageDescription` or `shortDescription`, and Open Graph `og:type` to `product` with the page URL and primary image.

File:



```
1 In src/utls/product-schema.ts:
2 generateProductSchema(product, productUrl)
```

Product Listing Page (PLP)

What Gets Emitted

The PLP emits a `schema.org/CollectionPage` with a `mainEntity` of type `ItemList`.

`CollectionPage`

- `name`: Category name
- `description`: `category.pageDescription`, when present
- `url`: Canonical category URL

`ItemList` (inside `mainEntity`)

- `numberOfItems`: Total from search result
- `itemListElement`: Up to 24 products, each as a `ListItem` wrapping a lightweight `Product` (name, url, image, offers)

Pricing

Each list item uses the lowest promotional price when promotions exist. Otherwise, it uses the base price. For master products, the schema considers variant-level prices.

Availability

The `orderable` field on the search hit drives availability. If the hit doesn't expose orderability but `config.search.products.refine.orderableOnly` is `true`, then the storefront infers `InStock` availability for all listed hits.

BreadcrumbList

Built from `parentCategoryTree` when present, plus the current category. All category URLs use `buildCategorySchemaUrl` to preserve site/locale prefixes.



the full first page of results, not just the above-the-fold slice.

SeoMeta

Sets `<title>` to the category name, meta description to `pageDescription` or the general description, and `og:type` to `website` with the canonical page URL.

File:



```
1 In src/utils/category-schema.ts:
2 generateCategorySchema({ category, searchResult, page
```

URL Construction

Storefront Next builds all JSON-LD URLs from the public storefront origin, not from `product.slugUrl` or internal API URLs. This approach is critical in serverless or proxied environments, such as Managed Runtime and content delivery networks (CDNs), where `request.url` can contain internal routing addresses.

File:



```
1 src/utils/schema-url.ts
```

Key functions:



```
1 // Reads x-forwarded-host / x-forwarded-proto headers
2 getPublicOrigin(request: Request): string
3
4 // Builds product and category URLs that preserve the
5 buildProductSchemaUrl({ productId, origin, currentPage
6 buildCategorySchemaUrl({ categoryId, origin, currentP
```

Example: Multisite URL Preservation



```
1 // Current page: https://example.com/global/en-GB/cat
2 // Built product URL:
3 buildProductSchemaUrl({
4   productId: "12345",
5   origin: "https://example.com",
6   currentPageUrl: "https://example.com/global/en-GB/c
7 });
8 // → 'https://example.com/global/en-GB/product/12345'
```



Rendering

Both PDP and PLP inject JSON-LD via the `JsonLd` component, which renders a `<script type="application/ld+json">` tag. The loader defers schema generation and returns an unresolved Promise. `JsonLd` is wrapped in `<Suspense>` using React's `use()` so that it streams with Server-Side Rendering (SSR) after product data is ready.

```
1 // JsonLd component usage (already wired in the route)
2 import { JsonLd } from "@components/json-ld";
3
4 <Suspense fallback={null}>
5   <JsonLdWrapper schemaPromise={loaderData.productSch
6 </Suspense>;
```

If `generateProductSchema` or `generateCategorySchema` throws, the error is logged in development and no script tag is rendered. The page continues to load normally.

Script IDs:

- PDP: `id="product-schema"`
- PLP: `id="category-schema"`

Customizing the Schema

Add Aggregate Ratings (PDP)

The `ProductSchema` type already includes an `aggregateRating` field. Wire it in `product-schema.ts` after fetching rating data from your review provider:

```
1 schema.aggregateRating = {
2   "@type": "AggregateRating",
3   ratingValue: "4.5",
4   reviewCount: "128",
5   bestRating: "5",
6   worstRating: "1",
7 };
```

Add FAQ Structured Data

If your PDP or editorial pages include FAQ content, add a `FAQPage` schema alongside the product schema:

```
1 <JsonLd
2   id="faq-schema"
3   data={{
4     "@context": "https://schema.org",
5     "@type": "FAQPage",
```



```
11         @type : Answer ,
12         text: "Available in XS-XXL.",
13     },
14 },
15 ],
16 }}
17 />
```

Enrich Catalog Fields

The quality of AEO/GEO signals depends directly on the data in your B2C Commerce catalog. Make sure that these Business Manager fields are populated for every product and category.

- `pageDescription`: Used as the meta description and JSON-LD description
- `longDescription` or `shortDescription`: Fallbacks for JSON-LD description
- `brand`: Populates `schema:brand`
- `ean`: Populates `schema:gtin`
- `manufacturerSKU`: Populates `schema:mpn`

Google's Rich Results Test flags structured data that contradicts visible on-page content as a quality issue. Keep catalog fields in sync with what shoppers see.

Validation

After making changes, validate with:

- [Google Rich Results Test](#) : Checks eligibility for rich snippets
- [Schema.org Validator](#) : Validates against the schema.org spec

In development, `JsonLd` logs errors to the console when invalid data is passed. These errors are suppressed in production.

See Also

- [SEO and Metadata](#): Canonical URLs, `hreflang`, full `SeoMeta` prop reference, query-parameter allowlists

DID THIS ARTICLE SOLVE YOUR ISSUE?

Let us know so we can improve!

[Share your feedback](#)



Try for free



MuleSoft

Tableau

Commerce Cloud

Lightning Design System

Einstein

Quip

Component Library

APIs

Trailhead

Sample Apps

AppExchange

Events and Calendar

Partner Community

Blog

Salesforce Admins

Salesforce Architects

© Copyright 2026 Salesforce, Inc. [All rights reserved.](#) Various trademarks held by their respective owners. Salesforce, Inc.
Salesforce Tower, 415 Mission Street, 3rd Floor, San Francisco, CA 94105, United States

[Legal](#)

[Terms of Service](#)

[Privacy Information](#)

[Responsible Disclosure](#)

[Trust](#)

[Contact](#)

[Use of Cookies](#)

[Cookie Preferences](#)



[Your Privacy Choices](#)

